Exploring Accessibility Trends and Challenges in Mobile App Development: A Study of Stack Overflow Questions

Amila Indika
University of Hawai'i at Mānoa
amilaind@hawaii.edu

Christopher Lee
University of Hawai'i at Mānoa
clee48@hawaii.edu

Haochen Wang
University of Hawai'i at Mānoa
hwang400@hawaii.edu

Justin Lisoway
University of Hawai'i at Mānoa
jlisoway@hawaii.edu

Anthony Peruma
University of Hawai'i at Mānoa
peruma@hawaii.edu

Rick Kazman
University of Hawai'i at Mānoa
kazman@hawaii.edu

Abstract

The proliferation of mobile applications (apps) has made it crucial to ensure their accessibility for users with disabilities. However, there is a lack of research on the real-world challenges developers face in implementing mobile accessibility features. This study presents a large-scale empirical analysis of accessibility discussions on Stack Overflow to identify the trends and challenges Android and iOS developers face. We examine the growth patterns, characteristics, and common topics mobile developers discuss. Our results show several challenges, including integrating assistive technologies like screen readers, ensuring accessible UI design, supporting text-to-speech across languages, handling complex gestures, and conducting accessibility testing. We envision our findings driving improvements in developer practices, research directions, tool support, and educational resources.

Keywords: mobile apps, Android, iOS, accessibility, stack overflow

1. Introduction

With billions of people currently using smartphones for tasks beyond communication—such as finance, health, and entertainment (Clearbridge-mobile, 2022)—it's essential for mobile application (app) developers to create apps that cater to a wide range of users, including the 1.3 billion people globally with disabilities ("World Health Organization", 2023).

Prior research has explored mobile app accessibility in multiple ways, such as analyzing app codebases for accessibility issues (da Silva et al., 2020; Eler et al., 2018), constructing accessibility taxonomies and guidelines (Ballantyne et al., 2018; El-Glaly et al., 2018), construction of tools (Siebra et al., 2018; Silva et al., 2018a), and examining user reviews for accessibility feedback (Reyes Arias et al., 2022). However, there is little research on the real-world challenges in implementing accessibility features. To this end, we conducted a large-scale empirical study of mobile accessibility discussions on Stack Overflow, mining 15 years of data. Stack Overflow is a leading programming Question-and-Answer website with millions of questions, answers, and users ("Stack Overflow Developer Survey", 2023).

1.1. Goals & Research Questions (RQs)

Employing both quantitative and qualitative analyses of question metrics and content, this study aims to understand how extensively mobile app developers seek assistance in incorporating accessibility features into their apps and the key trends, topics, and challenges therein. We address these objectives through the following research questions (RQs):

RQ1: How have mobile app accessibility questions on Stack Overflow grown over the years? This RQ aims to understand how often developers seek help from the community to make their mobile apps accessible. We achieve this by measuring the yearly growth of mobile accessibility questions on Stack Overflow.

RQ2: What are the characteristics of mobile app accessibility questions on Stack Overflow? This RQ involves analyzing the metadata associated with mobile accessibility questions. Specifically, we explore popular tags and gather statistical measures for attributes such as the score, views, answers, comments, and response times. By analyzing these metrics, we can gather insights into the level of engagement and the community's interest in mobile accessibility topics.

URI: https://hdl.handle.net/10125/109737
978-0-9981331-8-8

RQ3: What are the challenges associated with mobile app accessibility development? Through this RQ, we aim to identify the range of mobile accessibility challenges app developers discuss. We achieve this by combining natural language processing techniques and manual reviews.

The findings of this study can benefit various stakeholders. App developers can gain insights into common accessibility challenges and better prepare when building their apps. Researchers can use this data to advance the knowledge and understanding of mobile app accessibility. And accessibility advocates and organizations can tailor their support and resources to address the challenges identified.

2. Related Work

This section reports on research studies that have examined mobile app accessibility.

2.1. Stack Overflow Mining for Mobile Accessibility

Vendome et al. (2019) mined 810 Stack Overflow discussion threads and 13,817 GitHub Android repositories to assess developers' use of assistive technologies, such as Accessibility-APIs. They found that only 2.08% of apps imported accessibility APIs and were often used for non-accessibility purposes, like notification retrieval and touch interaction automation. Unlike their study, which focused solely on Android and accessibility APIs, our research examines a broader range of iOS and Android questions, focusing on mobile accessibility in general. Ma et al. (2022) analyzed dark mode accessibility issues in Android apps by examining posts and over 6,000 apps. They used Latent Dirichlet Allocation (LDA) to categorize developer challenges, summarizing accessibility issues specific to dark mode. However, our study focuses on accessibility issues beyond dark mode. Fontao et al. (2018) mined over 1.5 million questions related to Android, iOS, and Windows, categorizing discussions into hot topics and unanswered questions using LDA. Although they provided ten strategies for developer governance, our study differs by focusing solely on mobile app accessibility with a smaller dataset and excluding the Windows platform. Similarly, Rosen and Shihab (2016) analyzed over 13 million Stack Overflow questions to explore mobile development discussions, categorizing them into 40 categories using LDA topic modeling. They identified difficult-to-answer questions like connectivity, graphics, and media/streaming questions. However, none of these categories specifically addressed mobile accessibility. Lastly, Linares-Vásquez et al. (2013) explored Stack Overflow questions across various platforms like Android, iOS, BlackBerry, and Windows using LDA. The authors found that most developers answered questions on a single platform. Also, out of the 20 topics identified, none were related to mobile accessibility.

2.2. Repository Mining for Mobile Accessibility

Ross et al. (2020) analyzed 9,999 free Android apps, identifying seven accessibility barriers affecting impaired individuals. Although their research offers valuable insights, their dataset was not designed explicitly for accessibility analysis, and their approach does not compare with other platforms like iOS. In contrast, our study explores mobile accessibility across iOS and Android by analyzing Stack Overflow discussion posts, providing a broader view of developers' challenges. Di Gregorio et al. (2022) found that developers lack effective mechanisms to address mobile accessibility issues despite growing interest. Their study examined the implementation of accessibility standards, highlighting a lack of practical guidelines, a lack of developer awareness of disabilities, and technical challenges in implementing accessibility features. Unlike their focus on Android, our study covers iOS and Android and uses Stack Overflow data mining to explore accessibility issues. Acosta-Vargas et al. (2020) identified a significant gap in mobile developers' awareness of accessibility, revealing non-compliance with Web Content Accessibility Guidelines (WCAG) 2.1 in popular apps through manual and automated reviews. Similarly, Alshayban et al. (2020) studied over 1,000 Android apps, uncovering 11 common accessibility issues. They also surveyed Android developers to understand their perceptions of mobile accessibility. Their research attributes many of these issues to developer unawareness, additional costs, and insufficient support from management, highlighting the challenges disabled users face in finding accessible apps.

2.3. Automated Accessibility Detection and Evaluation Tools

Automated tools for detecting and repairing accessibility issues are well-documented in existing literature. For example, Alotaibi et al. (2021) developed a tool that refactors and repairs identified accessibility problems. Additionally, tools like Xbot (Chen et al. (2021)) focus on detecting accessibility issues, while dynamic test generation tools like MATE (Eler et al.

(2018)) uncover these issues during testing.

Existing research has utilized accessibility evaluation tools to assess mobile applications' compliance with accessibility guidelines (Mateus et al. (2021)). Additionally, current literature shows automated accessibility evaluation tools are developed specifically for mobile apps (Silva et al. (2018b)).

3. Method

Our research involved extracting Stack Overflow posts based on their tags and then analyzing the posts' metadata and content. As of August 2024, Stack Overflow contains approximately 1.42 million questions tagged with "Android" and 0.69 million tagged with "iOS," demonstrating the widespread appeal of this platform among mobile app developers. Our analysis included mixed-methods approach comprising quantitative and qualitative techniques. Quantitative techniques involved using standard statistical methods and applying Top2Vec (Angelov, 2020) topic modeling. Qualitative techniques involved a manual review of a statistically significant sample of the data by the authors. We will provide a detailed explanation of our analysis approach as we discuss each research question. We have made the study's Python scripts, SQL queries, shell scripts, and dataset available.[1].

3.1. Dataset

The data for this study was collected from Stack Overflow using the Stack Exchange Data Explorer[2] by querying the *Posts* table. In addition to the keyword *'accessibility'*, we conducted a snowballing analysis by reviewing tags associated with accessibility to identify additional keywords such as *'screen readers,' 'talkback,'* and *'voiceover,'*. This approach helped us gather more accessibility-related posts. We then used these newly identified keywords to perform additional queries to gather relevant data.

The extraction process involved several steps. First, we filtered accessibility-related questions by extracting post IDs using keywords in the title and tags. Next, we retrieved the question's body, metadata, and answers for the extracted question ID. Finally, we merged the questions and answers from each keyword-related query to create a comprehensive SQLite database of posts; we utilized the ID field to ensure no duplicate posts. The range of dates for the extracted posts was from January 2009 to October 2023. We extracted **6,371 posts**, comprising **3,022 questions** and **3,349 answers**. Each record in the database corresponds to a specific post and features 23 distinct fields.

[1]https://zenodo.org/doi/10.5281/zenodo.13753236
[2]https://data.stackexchange.com/stackoverflow

4. Results

RQ1: How have mobile app accessibility questions on Stack Overflow grown over the years?

Approach: RQ1 analyzed the temporal evolution of mobile accessibility challenges by examining the growth of mobile accessibility questions on Stack Overflow over the years. Specifically, we examined the yearly trends in questions without answers, questions without accepted answers, questions with accepted answers, and total questions.

Figure 1 shows this distribution. Out of the 3,022 mobile accessibility questions, 2,210 (73.13%) received answers, with 992 (32.83%) having accepted answers and 1,218 (40.30%) having non-accepted answers. Only 812 (26.87%) questions remained unanswered. The peak year was 2016, with 305 questions, while the median annual number of questions was 237, with a median of 60 unanswered questions.

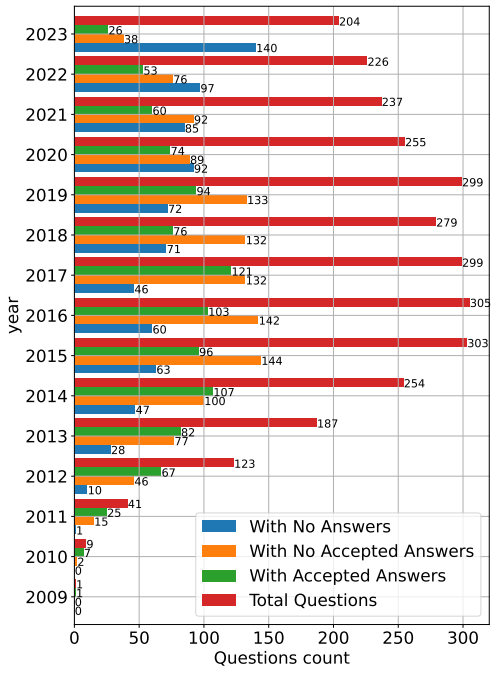

Figure 1: Mobile Accessibility questions asked by developers over the years

Focusing on Total Questions in Figure 1, the early 2010s saw increased discussion. This surge may have been driven by key developments like the introduction of VoiceOver for the iPhone 3GS (Apple-Press-Release, 2009) and Google's eyes-free project, which evolved

into TalkBack and was integrated into Android 4.0 in 2011[3,4]. From 2009 to 2014, our manual analysis identified 228 VoiceOver-related questions (≈7.5%), such as a question on integrating VoiceOver to support visually impaired users, as noted in a Stack Overflow question (SO, 2011). During the same period, there were 106 TalkBack-related questions (≈3.5%), including a post where a developer sought help with language support compatibility (SO, 2014a).

In 2011, Android 4.0 also introduced features, such as activating accessibility options through a clockwise rectangle touch gesture, Explore-by-Touch mode, and spoken feedback for screen touches. Developers, however, faced challenges with Explore-by-Touch, as noted in a Stack Overflow question (SO, 2012). These challenges are evidenced by over 20 questions (≈1%) related to explore-by-touch mode from 2009 to 2016. Additionally, Android 4.0 included system-wide font size adjustment, benefiting visually impaired users. Subsequent versions, such as Jelly Bean (4.1, 4.2, and 4.3)[5,6] introduced in 2012 and 2013, further improved accessibility features. For instance, version 4.1 introduced the accessibility focus feature for navigation using swipes and double taps, though developers encountered issues with this feature (SO, 2014b). Android 4.2 added screen magnification via triple-tapping, zoom-and-pan functionality, and enhanced Braille device feedback (Patel, 2012). The increased questions about gestures, like double taps, swipes, triple taps, and zooming—212 questions (≈7%) during this period—illustrate growing interest. Moreover, mobile app accessibility may have been influenced by regulations. For example, WCAG 2.0, introduced in 2008 (Caldwell et al., 2008), offered recommendations for making web content accessible, which mobile app developers also rely on (SO, 2017, 2019) demonstrate how WCAG is considered when integrating accessibility standards into their apps. Also, in 2015, the EN 301 549 European Standard (ETSI, 2018) aimed to conform ICT products and services in Europe with these accessibility requirements.

We observed a decline in mobile accessibility inquiries after 2016, which might be attributed to increased developer awareness and training. A 2022 survey by Level-Access (2022) found that 62.7% of 1,030 participants had received accessibility training, and 91% used free accessibility tools. This greater understanding and access to resources may have reduced the need for queries. Finally, the low number of mobile accessibility-related posts in 2009 and 2010 is likely due to Stack Overflow's inception in 2008 (Hanlon, 2013).

[3]https://code.google.com/archive/p/eyes-free

[4]https://developer.android.com/about/versions/android-4.0-highlights

[5]https://developer.android.com/about/versions/android-4.1#A11y

[6]https://developer.android.com/about/versions/jelly-bean

Summary for RQ1. Developers post questions to Stack Overflow when vendors introduce accessibility features, such as VoiceOver and TalkBack, into the operating system. Recently, the number of mobile app accessibility questions on the site has decreased. The enactment of accessibility regulations and the introduction of new accessibility features by the major vendors are likely factors influencing the growth of accessibility questions. Additionally, developing mature mobile accessibility practices reduces the need for repeated questions.

RQ2: What are the characteristics of mobile app accessibility questions on Stack Overflow?

Approach: RQ2 employs quantitative analysis to examine the characteristics of mobile accessibility questions and answers. First, we extracted tag occurrences in questions. Since our study focuses on mobile accessibility, we excluded the tags 'Accessibility,' 'Android,' and 'iOS' from this analysis as they did not yield significant insights, considering that these keywords were already involved in the data extraction process. Next, we calculated the first answer response time by retrieving the creation dates of questions and their first answers and computing the difference in days. Additionally, we queried the 'ViewCount,' 'AnswerCount,' 'Score,' and 'CommentCount' per question and counted the number of questions per user. Finally, we computed a statistical summary for these metrics.

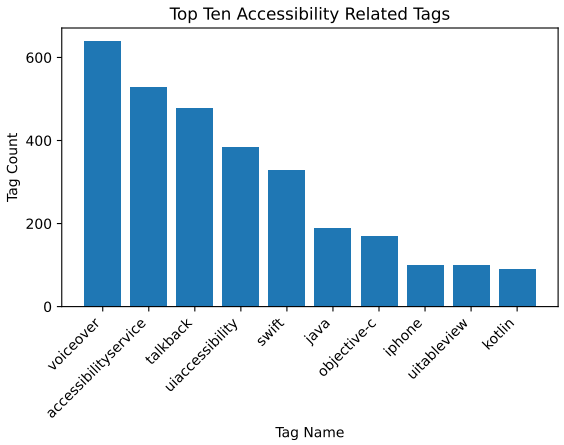

Figure 2: Top ten accessibility-related tags

Figure 2 shows the top ten accessibility-related tags. We observe that there is interest in screen readers

Table 1: Statistical measures of gathered Stack Overflow posts

Name	Average	Median	StdDev	Min	Max	Total
View Count per Post	1995.31	714.00	7595.86	7.00	340916.00	6029821.00
First Response Time in days per Question	99.73	1.28	320.17	0.00	3737.42	220403.22
Answer Count per Question	1.11	1.00	1.10	0.00	13.00	3349.00
Score per Question	2.61	1.00	6.66	-5.00	245.00	7895.00
Comment Count per Question	1.34	0.00	2.11	0.00	17.00	4054.00
Questions per User	1.27	1.00	0.97	1.00	26.00	3022.00

as evidenced by tags such as *voiceover* (i.e., the standard iOS screen reader), *uiaccessibility* (works with *voiceover*), *talkback* (i.e., the standard Android screen reader), and *accessibilityservice* (works with *talkback*). Additionally, *accessibilityservice* and *uiaccessibility* provide accessibility functionality beyond screen readers. Tags related to iOS development languages (*swift* and *objective-c*) and Android development languages (*java* and *kotlin*) were also prominent.

Table 1 reveals insights into mobile accessibility questions. Using the median, which is robust against outliers, we found that questions are typically answered within two days and receive at least one response, indicating higher community engagement. However, posts often have few comments and limited upvotes. Also, developers tend to ask more questions than they answer, primarily focusing on visual impairments rather than other disabilities.

Summary for RQ2.
Mobile accessibility questions on Stack Overflow typically receive responses within two days and at least one answer per query. Authors are more likely to post questions than answers, yet there are more answers than questions overall. Also, these questions generally lack comments and upvotes.

RQ3: What are the challenges associated with mobile app accessibility development?

Approach: In RQ3, we conduct a qualitative and quantitative analysis of mobile accessibility challenges using Top2Vec topic modeling (Angelov, 2020). The Top2Vec model clusters data to reveal insights into accessibility issues. Similar to Peruma et al. (2022), we preprocess question bodies using standard natural language processing techniques, including removing redundant spaces, expanding contractions, eliminating HTML tags and URLs, removing non-alphanumeric characters, tokenizing, converting text to lowercase, and filtering out stopwords. We also manually identified and removed custom stopwords. Then, we fed cleaned unigrams to the Top2Vec model for grouping questions. We manually optimized the Top2Vec model's hyperparameters, including $min\_cluster\_size$ and $min\_samples$ (McInnes et al., 2016), and employed the deep-learn option along with parallelized computations for efficiency. We inspected the model's segregation of unigram tokens to determine the ideal number of topics. With $min\_cluster\_size = 90$ and $min\_samples = 15$, the Top2Vec model generated meaningful clusters, resulting in seven topics.

After clustering, we reviewed a statistically significant random sample from each group (90% confidence interval, 10% error margin) for in-depth analysis. Table 2 details the number of questions per topic, those manually reviewed per topic, and the most frequent uni-grams per topic. Screen readers and navigation issues are the most common (18.76%), followed by custom Android accessibility service issues (18.66%). Thus, significant challenges include integrating screen readers and addressing navigation issues. Other typical problem areas include UI elements and interactions, multilingual support, and dynamic texts. Below, we describe each topic with an example.

Screen Readers and Navigation: Screen reader support is essential for making digital content accessible to users with visual and motor impairments (Lazar et al., 2007). Our manual inspection revealed that mobile developers often encounter challenges integrating screen readers with data visualizations like tables and collections. In particular, iOS developers face difficulties with VoiceOver integration for TableViews, CollectionViews, and ScrollViews, while Android developers have issues with Talkback integration for RecyclerViews. An example is shown in Quote 4.1.

UITableView within a UITableViewCell is not selectable by VoiceOver

"... Does anyone know how to make a UITableView inside of a UITableViewCell selectable by VoiceOver?"

Quote 4.1: An iOS developer requesting help with VoiceOver integration to UITableView (SO, 2022a)

Another common challenge is integrating screen

Table 2: Summary of question counts, reviewed questions, and representative words for each Top2Vec topic

Topic	Question Count	Question Percentage	Manually Reviewed	Associated Unigrams
Screen Readers and Navigation	567	18.76%	61	cells, cell, tableview, table, collection
Custom Android Accessibility Services: Configuration & Deployment	564	18.66%	61	manifest, service, package, permission, services
Accessibility of UI Elements and UI Interaction	456	15.09%	59	edittext, fragment, listview, compose, activity
Multilingual & Non-Standard Text, and Text-To-Speech (TTS) Support	410	13.57%	59	impaired, english, language, speech, tts
Touch gestures and User Interaction	397	13.14%	58	gestures, perform, touch, finger, fingers
Accessibility Testing, Troubleshooting, and Automation	327	10.82%	57	xcode, crash, inspector, uiaccessibility, simulator
Dynamic Text Size in UI Design	301	9.96%	56	font, size, dynamic, large, settings
Total	**3,022**	**100.00%**	**411**	————

readers with nested views or subviews. In addition, guiding impaired users to focus on specific accessibility components with screen readers is complex, requiring logical flow, consistency, contextual information, and efficient navigation. Developers also encounter challenges when implementing screen readers to follow specific gesture-based navigation patterns.

Custom Android Accessibility Services: Configuration & Deployment: Android developers face specific challenges when implementing custom accessibility services, particularly during their early learning phase. Issues often arise with permissions on particular brands/models and older devices and configuring accessibility services in the manifest file. Developers also struggle with enabling accessibility service options after a reboot. In addition, questions about compatibility and deployment across different platforms and debugging Android configurations are typical. An example is shown in Quote 4.2.

AccessibilityService stops receiving events on reboot

" ... If you reboot, you have to disable/re-enable the service from the accessibility services menu. Why does the app not get the events after a reboot?"

Quote 4.2: An Android developer asking help to re-enable AccessibilityService after reboot (SO, 2013)

Accessibility of UI Elements and UI Interaction: Developers frequently encounter issues providing accessibility for UI interactions and elements (e.g., text, buttons, image views, radio buttons, and text views), as shown in Quote 4.3. Challenges include text fields not updating correctly and redundant readings of hints and content descriptions, leading to confusion for users. Focus management problems and overlapping views like popups/dialogs further complicate navigation. Precise announcements of errors and labels and preventing repetitive auditory feedback are essential for enhancing UI accessibility. Moreover, fixing issues like misreading list items and improper focus on UI fragments is crucial to providing a seamless user experience.

Android Increase touch target of edit text as per accessibility without affecting design

"... If I add 14dp padding to top and bottom then the editText is as per accessibility but it is affecting design and increasing height of editText which I don't want. Is there any alternate solution to increase touch target with affecting design?"

Quote 4.3: An Android developer seeking help with setting accessibility for an EditText field (SO, 2022b)

Multilingual & Non-Standard Text, and Text-To-Speech (TTS) Support: Developers often struggle to support multiple languages like Spanish, Chinese, Tamil, German, and Hebrew, as illustrated in Quote 4.4. Interference between TalkBack and Text-To-Speech (TTS) services is also typical. In addition, reading non-standard text such as acronyms, special characters, and mathematical expressions poses difficulties. Contextual ambiguities, like acronyms and homonyms, can also affect TTS output. For example, "no of" can be read as either "number of" or "no of". Moreover, visually impaired developers face challenges while developing these accessibility features.

iOS voiceOver/accessibility foreign words pronuncation
"... The word 'Sound' spoken with German VoiceOver language doesn't make sense (it should say 'saund', but sounds like 'sund'). Is there a way to give voice-over information about a word's language?"

Quote 4.4: A developer having difficulty providing screen reader support for German. (SO, 2016)

Touch Gestures and User Interaction: Developers often struggle with handling multiple finger gestures, such as swipes, clicks, long presses, sliding, and zoom-and-panning, with accessibility services, as shown in Quote 4.5. Common issues include user interaction using gestures such as swiping through lists, controlling seek bars, and accessing keyboard overlays. Additional challenges involve implementing auto-clicking, magic tap with iOS, and screen reader support with explore-by-touch and double-taps.

How to dispatch multi touch gestures using AccessibilityService (disptachGesture)

"... I want to develop an app that dispatches complex gesture for user. ... how can I dispatch them using Accessibility Service (dispatchGesture) function."

Quote 4.5: An Android developer seeks assistance for integrating multi-touch gestures (SO, 2022c)

Accessibility Testing, Troubleshooting, and Automation:

OCMock - how to mock accessibility in iOS

"I am using xcode and my mocking frameworks is OCMock. How can i use OCMock to mock that accessibility is turned on so i can run some simple accessibility UI tests? ..."

Quote 4.6: An iOS developer facing an issue with accessibility UI tests (SO, 2015)

Developers often encounter challenges with accessibility testing and debugging tools, particularly iOS developers who struggle with tools such as Accessibility Inspector, Appium Inspector, XCTest, and KIF. Quote 4.6 provides an example. These challenges include integrating accessibility tests using XCTest and KIF, comparing accessibility elements between Xcode's Accessibility Inspector and Appium Inspector, and the complex nature of UI automation and testing for validating accessibility features. Additionally, there are challenges in cross-platform development using Xamarin and implementing custom actions like modifying configurations, inverting colors, or preventing VoiceOver from reading certain elements.

Dynamic Text Size in UI Design: Developers frequently face issues with dynamically adjusting text sizes based on accessibility settings, using tools like iOS's dynamic-type feature or Cascading Style Sheets (CSS) adjustments. They also struggle with retrieving and applying settings for font size and color adjustments. Platform-specific challenges commonly arise with Flutter, iOS, or Android APIs when implementing dynamic text size features. Maintaining UI integrity and aesthetics across elements like buttons, list views, and data grids is also problematic. Quote 4.7 illustrates an example of such a scenario.

Limit supported Dynamic Type font sizes

"I want to support Dynamic Type but only to a certain limit ... Is there an easy way to accomplish this with standard UITableViewCell styles?"

Quote 4.7: An iOS developer struggles with setting Dynamic Type font sizes (SO, 2014c)

Font-related issues are also a concern. Developers need to handle emphasis and ensure fonts scale dynamically. Integrating dynamic text size adjustments with other accessibility features, such as color and contrast adjustments, transparency, and touch gestures, adds another layer of complexity. Cross-platform frameworks like Xamarin and Flutter also present challenges for handling text size adjustments.

Summary for RQ3. Developers face challenges when implementing accessibility features. These include dynamically adjusting text sizes while maintaining UI integrity, conducting accessibility testing and debugging, supporting multiple languages and non-standard text for Text-To-Speech, handling complex touch gestures and user interactions, ensuring accessibility of UI elements and interactions, configuring and deploying custom Android accessibility services, and integrating screen readers with data visualizations and navigation.

5. Discussion

Our research highlights the need to prioritize accessibility in mobile app design and development, ensuring all users can fully engage with mobile technology and benefit equally. By analyzing 15 years of developer questions, we identified a broad spectrum of accessibility barriers, corroborating the insights of previous studies focused on specific app codebases.

Our findings are closely parallel with those of

Fontao et al. (2018), as both highlight Stack Overflow's popularity for discussing mobile app issues and observe accessibility questions decline post-2015. We also found that the median response time for mobile accessibility questions is under two days, consistent with their observation that most questions across various tags were answered within 48 hours. Top tags related to mobile accessibility, such as Swift, Java, and UITableView, appeared among the top 30 tags in Fontao et al.'s study. However, other accessibility tags were absent, likely due to our specific focus on mobile accessibility. Additionally, both studies suggest that official events influence question frequency.

Our RQ3 findings indicate that most mobile developers prioritize support for the visually impaired, often neglecting other impairments, similar to trends observed by Vendome et al. (2019). We found that many Android developers struggle with Accessibility Services, explaining the underutilization of Android Accessibility APIs noted by Vendome et al.

5.1. Theoretical Underpinnings

The Technology Acceptance Model (Davis, 1989) helps explain mobile developers' engagement with accessibility features by focusing on Perceived Ease of Use (PEOU) and Perceived Usefulness (PU). As observed in RQ3, developers recognize the value of accessibility in enhancing app quality and broadening the user base (high PU). However, they struggle with the technical challenges of implementation (low PEOU), often seeking community support. This hinders the adoption of accessibility best practices, even when developers understand their importance. Addressing low PEOU through better tools, documentation, and community support is essential to increasing the adoption of accessibility features in mobile apps.

5.2. Takeaways and Implications

Need for improved developer education and training on mobile accessibility. Our findings reveal that developers face significant challenges incorporating accessibility features, resulting in low PEOU. This presents an opportunity for educators to offer specialized mobile app courses or training on accessibility, catering to developers of all skill levels. Organizations should also prioritize accessibility-focused training for their developers, which could ultimately increase PEOU and lead to greater adoption of accessibility features.

Capturing accessibility requirements early in the software development lifecycle. Developers and business analysts should explicitly define accessibility requirements during the requirements-gathering stage to ensure the app supports necessary disabilities. This understanding allows architects and designers to integrate accessibility into the app's architecture and UI from the outset. This approach allows project managers to involve accessibility specialists early or provide relevant training, minimizing the need for rework later in the development lifecycle. Additionally, insights from our RQ3 findings can help teams anticipate and address common accessibility challenges proactively.

Advancing accessibility tools through collaborations between tool vendors and researchers. Our findings, especially from RQ3, illustrate the need for more advanced and user-friendly accessibility tools. Collaboration between academic researchers and tool vendors could drive the creation of intuitive, innovative, practical, and scalable solutions that integrate seamlessly into developers' workflows, making accessibility implementation more efficient. Additionally, accessibility testing frameworks (e.g., Axe) and libraries that simplify accessible component integration can enhance PEOU. When developers are familiar with these tools, they are more likely to adopt accessibility practices.

6. Threats To Validity

Although other sources exist for mobile app developer discussions (e.g., Reddit, Discord, GitHub, etc.), our study is limited to Stack Overflow, a popular programming Q&A website. We constructed our query using specific terms and limited it to Android and iOS. Hence, even though we extracted 3,022 questions, there is still room for further analysis and expansion. Our qualitative analysis involves a manual review of a statistically significant sample of data. Even so, there is a chance that the reviewed sample might not include all scenarios present in the dataset. Additionally, as our study is scoped to the topics and challenges in mobile app accessibility, our analysis is only limited to questions and not answers, which can be analyzed in future studies. Finally, our findings provide a point-in-time snapshot of the mobile accessibility challenges developers encounter.

7. Conclusion & Future Work

As mobile apps become integral to daily life, developers must ensure they are accessible to users with disabilities. This study analyzes over 15 years of Stack Overflow questions, revealing trends and challenges in mobile app accessibility. The findings show a rise in accessibility questions, often coinciding

with the introduction of new mobile accessibility features in Android and iOS. Most accessibility questions receive answers within two days. Key challenges include integrating screen readers, managing custom accessibility services, ensuring accessible UI elements and interactions, supporting multi-language text-to-speech, implementing complex gestures, and conducting accessibility testing.

The insights from this study highlight the importance of considering accessibility throughout the lifecycle, not just as an afterthought. Educators and advocates can use these findings to connect academic curricula with real-world accessibility challenges. Furthermore, our findings can guide the formulation of new guidelines, best practices, and resources to tackle the critical challenges confronting developers.

Our future work includes surveying professional mobile app developers to validate this study's findings. We will take two approaches: conducting case studies with organizations that build mobile apps to understand their accessibility practices and launching a large-scale online survey to gather insights on accessibility challenges. We plan to identify survey participants through platforms like LinkedIn.

References

Acosta-Vargas, P., Salvador-Ullauri, L., Jadán-Guerrero, J., Guevara, C., Sanchez-Gordon, S., Calle-Jimenez, T., Lara-Alvarez, P., Medina, A., & Nunes, I. L. (2020). Accessibility assessment in mobile applications for android. *Advances in Human Factors and Systems Interaction*.

Alotaibi, A. S., Chiou, P. T., & Halfond, W. G. (2021). Automated repair of size-based inaccessibility issues in mobile applications. *2021 36th IEEE/ACM International Conference on Automated Software Engineering (ASE)*.

Alshayban, A., Ahmed, I., & Malek, S. (2020). Accessibility issues in android apps: State of affairs, sentiments, and ways forward. *Proceedings of the ACM/IEEE 42nd International Conference on Software Engineering*.

Angelov, D. (2020). Top2vec: Distributed representations of topics. *arXiv preprint*. https://doi.org/10.48550/arXiv.2008.09470

Apple-Press-Release. (2009). Apple announces the new iphone 3gs-the fastest, most powerful iphone yet. https://www.apple.com/newsroom/2009/06/08Apple-Announces-the-New-iPhone-3GS-The-Fastest-Most-Powerful-iPhone-Yet/

Ballantyne, M., Jha, A., Jacobsen, A., Hawker, J. S., & El-Glaly, Y. N. (2018). Study of accessibility guidelines of mobile applications. *Proceedings of the 17th international conference on mobile and ubiquitous multimedia*.

Caldwell, B., Cooper, M., Reid, L. G., Vanderheiden, G., Chisholm, W., Slatin, J., & White, J. (2008). Web content accessibility guidelines 2.0. (dec 2008). https://www.w3.org/TR/WCAG20/

Chen, S., Chen, C., Fan, L., Fan, M., Zhan, X., & Liu, Y. (2021). Accessible or not? an empirical investigation of android app accessibility. *IEEE Transactions on Software Engineering*, *48*(10), 3954–3968. https://doi.org/10.1109/TSE.2021.3108162

Clearbridge-mobile. (2022). 50 mobile app statistics for app growth in 2022. https://clearbridgemobile.com/stats-for-mobile-app-growth-and-success/

da Silva, H. N., Endo, A. T., Eler, M. M., Vergilio, S. R., & Durelli, V. H. S. (2020). On the relation between code elements and accessibility issues in android apps. *Proceedings of the 5th Brazilian Symposium on Systematic and Automated Software Testing*.

Davis, F. D. (1989). Perceived usefulness, perceived ease of use, and user acceptance of information technology. *MIS quarterly*, 319–340.

Di Gregorio, M., Di Nucci, D., Palomba, F., & Vitiello, G. (2022). The making of accessible android applications: An empirical study on the state of the practice. *Empirical Software Engineering*, *27*(6), 145. https://doi.org/10.1007/s10664-022-10182-x

Eler, M. M., Rojas, J. M., Ge, Y., & Fraser, G. (2018). Automated accessibility testing of mobile apps. *2018 IEEE 11th International Conference on Software Testing, Verification and Validation (ICST)*, 116–126. https://doi.org/10.1109/ICST.2018.00021

El-Glaly, Y. N., Peruma, A., Krutz, D. E., & Hawker, J. S. (2018). Apps for everyone: Mobile accessibility learning modules. *ACM Inroads*, *9*(2), 30–33. https://doi.org/10.1145/3182184

ETSI. (2018). Etsi en 301 549 - v1.1.2. https://www.etsi.org/deliver/etsi_en/301500_301599/301549/01.01.02_60/en_301549v010102p.pdf

Fontao, A., Ábia, B., Wiese, I., Estácio, B., Quinta, M., Santos, R. P. d., & Dias-Neto, A. C. (2018). Supporting governance of mobile application developers from mining and analyzing technical questions in stack overflow. *Journal*

of Software Engineering Research and Development, *6*.

Hanlon, J. (2013). Five years ago, stack overflow launched. then, a miracle occurred. https://stackoverflow.blog/2013/09/16/five-years-ago-stack-overflow-launched-then-a-miracle-occurred/

Lazar, J., Allen, A., Kleinman, J., & Malarkey, C. (2007). What frustrates screen reader users on the web: A study of 100 blind users. *International Journal of human-computer interaction*, *22*(3), 247–269.

Level-Access. (2022). 2022 state of digital accessibility - levelaccess.com. https://www.levelaccess.com/wp-content/uploads/2022/10/2022_State_of_Digital_Accessibility_Report.pdf

Linares-Vásquez, M., Dit, B., & Poshyvanyk, D. (2013). An exploratory analysis of mobile development issues using stack overflow. *2013 10th Working Conference on Mining Software Repositories (MSR)*, 93–96.

Ma, S., Chen, C., Khalajzadeh, H., & Grundy, J. (2022). A first look at dark mode in real-world android apps. *ACM Transactions on Software Engineering and Methodology*.

Mateus, D. A., Silva, C. A., de Oliveira, A. F., Costa, H., & Freire, A. P. (2021). A systematic mapping of accessibility problems encountered on websites and mobile apps: A comparison between automated tests, manual inspections and user evaluations. *Journal on Interactive Systems*, *12*(1).

McInnes, L., Healy, J., & Astels, S. (2016). Parameter selection for hdbscan*. https://hdbscan.readthedocs.io/en/latest/parameter_selection.html

Patel, N. (2012, October). Android 4.2 adds gesture typing, wireless tv display, multiple user support on tablets, and more - the verge.

Peruma, A., Simmons, S., AlOmar, E. A., Newman, C. D., Mkaouer, M. W., & Ouni, A. (2022). How do i refactor this? an empirical study on refactoring trends and topics in stack overflow. *Empirical Software Engineering*, *27*(1), 11. https://doi.org/10.1007/s10664-021-10045-x

Reyes Arias, J. E., Kurtzhall, K., Pham, D., Mkaouer, M. W., & Elglaly, Y. N. (2022). Accessibility feedback in mobile application reviews: A dataset of reviews and accessibility guidelines. *Extended Abstracts of the 2022 CHI Conference on Human Factors in Computing Systems*. https://doi.org/10.1145/3491101.3519625

Rosen, C., & Shihab, E. (2016). What are mobile developers asking about? a large scale study using stack overflow. *Empirical Software Engineering*, *21*, 1192–1223.

Ross, A. S., Zhang, X., Fogarty, J., & Wobbrock, J. O. (2020). An epidemiology-inspired large-scale analysis of android app accessibility. *ACM Transactions on Accessible Computing (TACCESS)*. https://doi.org/10.1145/3348797

Siebra, C. A., Correia, W., Penha, M., Macêdo, J., Quintino, J., Anjos, M., Florentin, F., Silva, F. Q. B., & Santos, A. L. M. (2018). An analysis on tools for accessibility evaluation in mobile applications. *Proceedings of the XXXII Brazilian Symposium on Software Engineering*.

Silva, C., Eler, M. M., & Fraser, G. (2018a). A survey on the tool support for the automatic evaluation of mobile accessibility. *Proceedings of the 8th International Conference on Software Development and Technologies for Enhancing Accessibility and Fighting Info-Exclusion*.

Silva, C., Eler, M. M., & Fraser, G. (2018b). A survey on the tool support for the automatic evaluation of mobile accessibility. *Proceedings of the 8th International Conference on Software Development and Technologies for Enhancing Accessibility and Fighting Info-exclusion*.

SO. (2011). www.stackoverflow.com/q/5519923

SO. (2012). www.stackoverflow.com/q/12943149

SO. (2013). www.stackoverflow.com/q/16513327

SO. (2014a). www.stackoverflow.com/q/23710512

SO. (2014b). www.stackoverflow.com/q/26265970

SO. (2014c). www.stackoverflow.com/q/26371024

SO. (2015). www.stackoverflow.com/q/31701560

SO. (2016). www.stackoverflow.com/q/40948714

SO. (2017). https://stackoverflow.com/q/47737275/

SO. (2019). https://stackoverflow.com/q/55067524/

SO. (2022a). www.stackoverflow.com/q/72411194

SO. (2022b). www.stackoverflow.com/q/70821790/

SO. (2022c). www.stackoverflow.com/q/73134469

Stack overflow developer survey. (2023). https://survey.stackoverflow.co/2023/#community-stack-overflow-site-use

Vendome, C., Solano, D., Liñán, S., & Linares-Vásquez, M. (2019). Can everyone use my app? an empirical study on accessibility in android apps. *2019 IEEE International Conference on Software Maintenance and Evolution*. https://doi.org/10.1109/ICSME.2019.00014

World health organization. (2023). https://www.who.int/news-room/fact-sheets/detail/disability-and-health